

基于冻结 DINOv2 特征重建的工业图像异常检测技术报告

摘要

本文介绍一个面向工业图像异常检测与定位的离线方案。该方案以本地保存的 DINOv2 `vit_base_patch14_reg4` 为特征编码器，冻结编码器参数，仅训练轻量的瓶颈网络和 Transformer 解码器。训练阶段，模型学习重建正常图像的多层特征；推理阶段，以编码特征和重建特征之间的余弦距离构建连续的像素级异常图，并由高响应区域汇聚得到图像级异常分数。

项目采用单一共享模型，可随训练 manifest 中实际出现的类别集合训练和保存模型，而不依赖固定类别名称。训练与推理均通过命令行接收数据、manifest、模型和输出路径；预训练权重、模型权重及分数归一化参数均以本地文件形式加载，从而支持断网环境下运行。推理为每一个 `sample_id` 输出 `[0, 1]` 范围的图像级分数，以及与原图同尺寸的单通道 16-bit PNG 像素异常图。

实验共设计 5 组。由于具体测试数据尚待补充，本文已预留 Pixel-level F1 / Average Precision、Image-level F1 / Average Precision 和 Pixel-level AUROC 的填写位置，后续只需填入相应数值和实验配置即可。

关键词： 工业异常检测；无监督学习；DINOv2；特征重建；像素级定位

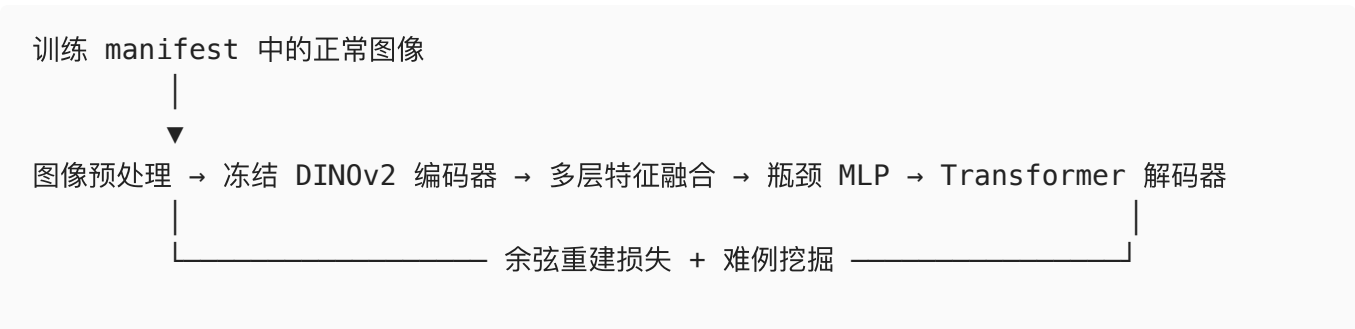
1. 问题定义与方案概述

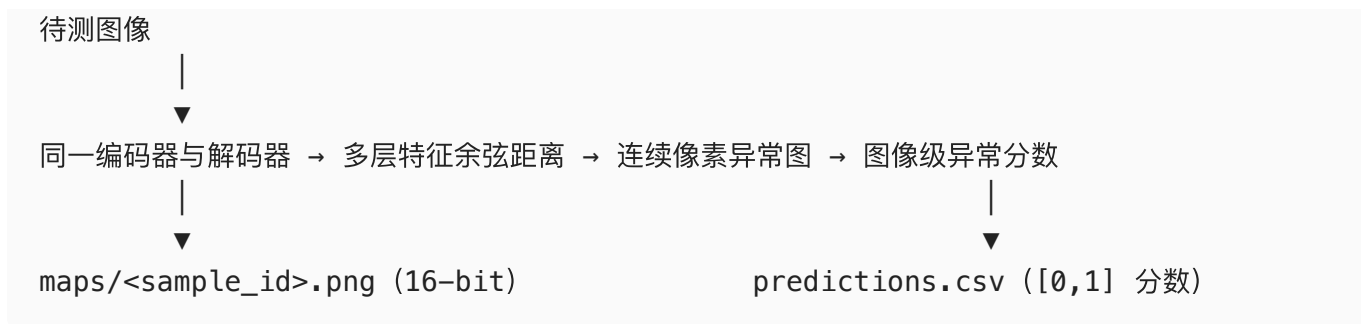
工业视觉检测既要判定一张图像是否存在异常，也要定位异常所在区域。设输入图像为 x ，模型输出：

- 图像级异常分数 $s(x) \in [0, 1]$ ，数值越大表示图像越可能异常；
- 像素级异常图 $A(x) \in [0, 1]^{H \times W}$ ，其中 (H, W) 与原始输入图像的尺寸一致。

本方案属于特征重建式异常检测。在通常的无监督设定中，训练 manifest 由正常图像构成，模型据此学习正常外观和结构表征；当输入图像含有未见异常时，异常位置更难被准确重建，编码特征与解码特征之间的差异会升高，从而形成异常响应。项目代码本身按 manifest 读取训练图像，不在源码中通过标签字段验证样本是否为正常样本。

整体流程如下：





2. 数据接口与预处理

2.1 Manifest 驱动的数据读取

项目通过 `src/core/manifest.py` 读取 CSV manifest。训练时读取 `category` 与 `image_path`；推理时还要求 `sample_id`。所有图像路径均相对于命令行传入的 `--data-root` 解释，因此实现不依赖数据集的绝对路径、目录扫描结果或类别列表。

`src/data/dataset.py` 按 manifest 的每一行读取图像。为兼容中文及其他非 ASCII 路径，图像读取使用 `np.fromfile` 与 `cv2.imdecode`，随后由 BGR 转为 RGB。推理数据集会额外记录原始高度和宽度，以便将模型输出的异常图恢复到原始图像尺寸。

2.2 图像变换

默认变换由 `src/data/transforms.py` 定义，训练与推理使用同一流程：

1. 将 RGB 图像转换为 `[0,1]` 的 float 张量；
2. Resize 至 `x`；
3. 中心裁剪至 `x`；
4. 使用 ImageNet 的均值 `(0,0,00)` 和标准差 `(0,0,0)` 归一化。

统一的预处理使训练和推理的特征分布保持一致。输出时，`src/engine/predict_engine.py` 会将像素异常图线性插值回原图大小，而不是将输入图像固定输出为裁剪尺寸。

3. 方法设计

3.1 冻结的 DINOv2 特征编码器

编码器为 `vit_base_patch14_reg4_dinov2`，预训练权重文件为 `dinov2_vitb14_reg4_pretrain.pth`。默认配置位于 `configs/default.json`，其核心设置为：

模块	设置
主干网络	DINOv2 ViT-Base, patch size 为 14
特征维度 / 注意力头数	768 / 12

模块	设置
取用的编码器层	blocks.2 至 blocks.9，共 8 层
主干参数	全部冻结
计算精度	float32

`src/models/dinomaly/model.py` 在建模后将所有参数冻结，再仅解冻 `bottleneck` 和 `decoder`。这一设计保留通用预训练视觉特征，同时避免在仅含正常样本的训练条件下对大规模主干进行过拟合，并降低训练所需的可学习参数量。

3.2 特征重建网络

模型主体定义于 `src/models/dinomaly/torch_model.py`。编码器输出的多层 token 特征先按层分组取均值融合，再经一个 MLP 瓶颈和 Transformer 解码器处理。

- **瓶颈层：**由 `DinomalyMLP` 实现，隐藏层维度为 $\times$ ，激活函数为 GELU，默认 dropout 为 0.3。
- **解码器：**默认深度为 6，每层为 `DecoderViTBlock`。注意力模块使用 `LinearAttention`：先对 $\cdot$ 施加 $()^1$ 映射，再计算线性复杂度的注意力聚合。
- **层间融合：**默认将编码器和解码器的 8 组特征分别划分为前 4 层、后 4 层两组，并在每组内取均值，以得到用于监督和推理的两组空间特征。

对于包含类别 token 与 register token 的特征序列，当前默认设置不直接将其用于空间异常图。实现依据图像 patch 数取最后 $H \times W$ 个 token，并将其还原为二维特征图，从而使异常定位对应图像空间位置。

3.3 训练目标：余弦重建与难例挖掘

对第 i 组编码器空间特征 E_i 与对应的解码器特征 D_i ，基础重建误差采用余弦距离：

$$1 - \cos(E_i, D_i)$$

训练损失由 `src/models/dinomaly/components/loss.py` 中的 `CosineHardMiningLoss` 实现。该损失对各特征组的整体余弦损失取平均，并在反向传播时降低易重建位置的梯度贡献。具体而言，训练开始后的前 1000 步逐步将降梯度比例从 0 提升至 0.9；被识别为易重建的位置其梯度乘以 0.1，其余较难位置保留原梯度。这样可使训练更关注难以重建的特征位置，减少解码器对异常模式的过度重建能力。

3.4 像素级异常定位与图像级判别

推理时，模型对每组特征计算空间余弦距离并插值至输入尺寸：

$$A = (1 - \cos(E_i, D_i)),$$

再对所有特征组平均，得到连续异常图：

$$A = \frac{1}{1} A$$

该异常图作为定位结果返回。为得到更稳健的图像级分数，代码将异常图缩放到 $\times$ ，施加核大小为 5、的高斯平滑，再取响应最高的 1% 像素的均值：

$$s = (1((A)))$$

训练结束后，`src/engine/train_engine.py` 会在训练 manifest 上统计 s 的最小值和最大值并保存为 `auxiliary/thresholds/minmax.json`。推理阶段依据

$$s \left(s \text{ ss } s, 0, 1 \right)$$

将图像分数归一化至 `[0,1]`。对应实现位于 `src/utils/normalization.py`。

4. 训练策略与实现配置

4.1 默认超参数

下表列出 `configs/default.json` 中的默认训练设置。

项目	默认值
batch size	8
最大训练步数	4000
优化器	StableAdamW
初始 / 基础学习率	0.002
最终学习率	0.0002
warmup 步数	100
AdamW betas	(0.9, 0.999)
权重衰减	0.0001
优化器内部裁剪阈值	1.0
全局梯度范数裁剪	0.1
随机种子	2026

优化器和调度器定义于 `src/models/dinomaly/components/optimizer.py`。StableAdamW 在 AdamW 更新中引入基于 RMS 的更新幅度缩放；学习率采用先 warmup 后余弦退火的 WarmCosineScheduler。

4.2 模型保存与加载

训练入口 `src/train.py` 调用 `src/engine/train_engine.py`，在 `--output-dir` 下生成完整的模型包：

```
<output-dir>/
├─ shared.pth
├─ model_manifest.json
└─ auxiliary/
    ├─ pretrained/dinov2_vitb14_reg4_pretrain.pth
    └─ thresholds/minmax.json
```

其中 `shared.pth` 保存 `state_dict`、模型配置和随机种子；`model_manifest.json` 保存模型模式、权重文件名、训练 manifest 中实际出现的类别、分数范围及模型结构参数。推理入口 `src/predict.py` 通过 `src/models/builder.py` 从 `--model-dir` 加载上述文件，不依赖硬编码的本地模型绝对路径。

5. 推理输出与接口适配说明

5.1 推理流程

`src/engine/predict_engine.py` 严格按推理 manifest 逐条处理样本，并使用原始 `sample_id` 命名输出文件。模型切换至 `eval()` 模式，且推理循环位于 `torch.inference_mode()` 内。若某一输入、模型文件或输出发生错误，异常会向上抛出并使程序非零退出，而不会静默跳过样本。

输出目录结构如下：

```
<output-dir>/
├─ predictions.csv
└─ maps/
    └─ <sample_id>.png
```

`predictions.csv` 包含 `sample_id`, `image_score` 两列。`image_score` 经过 min-max 归一化与截断，取值在 `[0,1]` 内。对于每一幅输入图像，`maps/<sample_id>.png` 均由 `src/utils/image_io.py` 写为单通道 uint16 PNG：异常图先截断到 `[0,1]`，再乘以 65535 并四舍五入；其宽高与原始输入图像一致。

5.2 交付约束的对应关系

下表说明的是当前项目实现与评测接口的对应关系，不将接口文件中的要求作为模型效果或实验结果的事实依据。

交付关注点	项目中的实现依据
路径由命令行传入	<code>src/core/args.py</code> 使用 <code>argparse</code> 接收 <code>--data-root</code> 、 <code>--manifest</code> 、 <code>--output-dir</code> 、 <code>--device</code> 、 <code>--num-workers</code> ；训练额外接收 <code>--seed</code> ，推理额外接收 <code>--model-dir</code> 。
按清单处理数据	<code>src/core/manifest.py</code> 解析 <code>manifest</code> ，训练与推理均由 <code>ManifestDataset</code> 按行读取相对路径。
共享模型与类别适配	<code>model_manifest.json</code> 的 <code>model_mode</code> 为 <code>shared</code> ；训练时从 <code>manifest</code> 动态收集并保存类别集合。
离线模型加载	编码器权重、 <code>shared.pth</code> 与 <code>minmax.json</code> 从模型目录加载；主干创建时显式使用 <code>pretrained=False</code> ，不在运行期下载权重。
连续定位分数	模型输出余弦距离异常图，保存时为 16-bit 连续灰度 PNG，而非二值掩膜。
图像级分数范围	<code>normalize</code> 将输出裁剪至 <code>[0,1]</code> ，并以 <code>sample_id</code> 写入 CSV。
可复现设置	<code>src/core/seed.py</code> 设置 Python、NumPy、PyTorch 和可用 CUDA 的随机种子；本报告采用种子 2026 记录实验。

需要说明的是，当前 `set_seed` 默认未主动开启 cuDNN 的确定性开关。因此，若在不同硬件、CUDA/cuDNN 或依赖版本下复现，仍可能存在少量数值差异；复现实验应固定软件环境、硬件条件和随机种子。

6. 实验

6.1 实验目的

实验从两个层面验证方案：其一，评估模型对异常图像的识别能力；其二，评估模型对缺陷区域的像素级定位能力。进一步地，比较六个参数版本的结果，考察 `use_context_recentering` 设置与模型版本变化带来的影响，并选择适合最终提交的模型。

根据提供的评测记录，版本 1 至版本 6 均完成了同一评测任务。最终采用**版本 4**；其模型超参数完全使用项目 `configs/default.json` 中的默认设置。

6.2 数据集与数据划分

本次训练采用仅含正常图像的训练集；评测集同时包含正常图像和缺陷图像。评测标注包括图像级正常/异常标签和像素级缺陷掩膜，因此可同时计算图像级与像素级指标。训练集的样本总数未在现有评测记录中单独给出，本文不据此臆测其数值。

项目	设置或统计
类别数	30
训练集	仅正常图像
评测集	正常图像 + 缺陷图像
评测集正常样本数	773
评测集缺陷样本数	1439
评测集图像总数	2212
标注	图像级标签 + 像素级缺陷掩膜
类别分布	存在不均衡：各类别正常样本数为 2–102，缺陷样本数为 2–240

表中的评测集统计来自各版本结果中的 ALL 行。类别不均衡意味着仅报告将全部样本汇总后的结果并不充分，因此本节同时报告全局（Global）与按类别宏平均（Macro Average）结果。全局统计将所有评测图像/像素直接汇总；宏平均则先在每个类别内计算指标，再对 30 个类别取算术平均。

6.3 实验环境与实现细节

实验运行于 NVIDIA RTX 5060 GPU 和 Intel Ultra 7 255HX CPU 环境，使用 Python 3.14 与 CUDA 13.0。依赖版本遵循项目 requirements.lock，其中 PyTorch 为 torch==2.13.0、TorchVision 为 torchvision==0.28.0。除版本 4 已明确采用项目默认配置外，其余版本的完整差异未在现有材料中逐项记录，故下表仅陈述可由配置文件或已提供信息确认的设置。

项目	最终版本 4 的设置
编码器	vit_base_patch14_reg4_dinov2
编码器权重	model/auxiliary/pretrained/dinov2_vitb14_reg4_pretrai
可训练模块	bottleneck 与 decoder；编码器冻结
特征层	默认 blocks.2 至 blocks.9
输入变换	Resize 至 ×，中心裁剪至 ×，ImageNet 归一化
bottleneck dropout / decoder depth	0.3 / 6
remove_class_token	false
use_context_recentering	false
精度	float32
batch size / 最大训练步数	8 / 4000
优化器	StableAdamW， 000, betas=(0.9, 0.999), weight decay=0.000

项目	最终版本 4 的设置
学习率调度	100 步 warmup，随后余弦退火至 0.0002，总计 4000 步
梯度裁剪	训练循环全局范数 0.1；优化器内部阈值 1.0
随机种子	2026

6.4 评价指标

评测从图像级判别和像素级定位两方面进行。给定连续图像分数或连续像素异常图，AP 衡量 Precision–Recall 曲线下的面积，AUROC 衡量不同阈值下的排序区分能力。F1 在所有有效阈值中取最大值，因此以下报告明确写为 **F1-max**，并同步给出取得该值的阈值；这避免将最优阈值 F1 与任意固定阈值 F1 混淆。

评价维度	指标	含义	
图像级	Image AUROC	正常图像与异常图像的排序区分能力	
图像级	Image AP	异常图像识别的 PR 表现	
图像级	Image F1-max	最优阈值下的图像级异常判别能力	
像素级	Pixel AUROC	正常与异常像素的排序区分能力	
像素级	Pixel AP	连续异常图对缺陷像素的定位质量	
像素级	Pixel F1-max	最优阈值下的二值缺陷区域定位能力	

6.5 主实验结果：最终版本 4

版本 4 使用项目默认超参数，是最终采用的模型。表中 Global 为所有评测样本及像素汇总后的结果，Macro Average 为逐类别计算后在 30 个类别上的宏平均。数值均保留至 4 位小数。

统计口径	Image AUROC	Image AP	Image F1-max		Pixel AUROC	Pixel AP	Pixel F1-max	$\mathcal{X}$
Global	0.8398	0.9096	0.8311	0.3038	0.8336	0.1190	0.2139	0.1606
Macro Average	0.8755	0.9193	0.8991	0.5846	0.8696	0.1987	0.2787	0.1347

从 Global 结果看，版本 4 的 Image AP 为 0.9096，表明其对异常图像具有较好的排序和判别能力。Pixel AP 为 0.1190、Pixel F1-max 为 0.2139，说明模型能够输出可用于定位的连续异常响应，但像素级精确定位仍明显比图像级判别更具挑战。Macro Average 下，版本 4 的 Image AUROC、Image F1-max、Pixel AUROC 和 Pixel AP 分别为 0.8755、0.8991、0.8696 和 0.1987，表明其在按类别平衡统计时仍保持稳定表现。

6.6 参数版本对比与消融观察

提供的评测截图包含版本 1 至版本 6，因此本报告保留全部六组记录。当前可确认的配置分组为：版本 1–4 未启用 use_context_recentering，版本 5–6 启用了该选项；版本 4 同时为项目默认配置。除该开关外，现有材料没有逐项列出各版本的其他参数差异，故不能把任意两个版本之间的差异严格归因于单一因素。

6.6.1 全局结果对比

版本	use_context_recentering	Image AUROC	Image AP	Image F1-max	Pixel AUROC	Pixel AP	Pixel F1-max
V1	false	0.8350	0.8960	0.8328	0.8322	0.1152	0.2033
V2	false	0.8373	0.9078	0.8317	0.8292	0.1153	0.2035
V3	false	0.8608	0.9288	0.8241	0.8306	0.0845	0.1694
V4 (最终)	false	0.8398	0.9096	0.8311	0.8336	0.1190	0.2139
V5	true	0.8601	0.9282	0.8251	0.8336	0.0894	0.1799
V6	true	0.8625	0.9304	0.8272	0.8327	0.0907	0.1844

在六个版本中，V6 的 Image AUROC 和 Image AP 最高，分别为 0.8625 和 0.9304；但 V4 的 Pixel AP 和 Pixel F1-max 最高，分别为 0.1190 和 0.2139，且 Pixel AUROC 与最优值并列 (0.8336)。由于像素级 F1/AP 是主要考核指标，最终选择 V4 而不是仅图像级指标更高的 V6。这一选择体现了图像级异常识别与像素级精细定位之间的权衡。

6.6.2 宏平均结果对比

版本 1 和版本 2 的测试未包含 AVG 行，故下表仅报告有完整宏平均记录的版本 3–6。

版本	Image AUROC	Image AP	Image F1-max	Pixel AUROC	Pixel AP	Pixel F1-max
V3	0.8645	0.9219	0.8966	0.8567	0.1961	0.2823
V4 (最终)	0.8755	0.9193	0.8991	0.8696	0.1987	0.2787
V5	0.8672	0.9236	0.8952	0.8581	0.1980	0.2833
V6	0.8718	0.9253	0.8985	0.8583	0.1977	0.2834

宏平均结果显示，V4 在 Image AUROC、Image F1-max、Pixel AUROC 和 Pixel AP 上为表中最高，V6 则在 Image AP 和 Pixel F1-max 上略高。结合全局统计，V4 的优势集中在像素级 AP、像素级 AUROC 及整体像素 F1-max，因此被确定为最终版本；V6 的结果则提示，后续可以在保持图像级识别优势的同时继续优化定位阈值或异常图后处理。

6.6.3 use_context_recentering 的观察

在现有结果中，未启用该开关的 V4 相比启用该开关的 V5/V6，取得更高的全局 Pixel AP (0.1190, 对比 0.0894/0.0907) 和 Pixel F1-max (0.2139, 对比 0.1799/0.1844)；而 V5/V6 的图像级 AP 更高。宏平均下，V4 的 Pixel AP 略高于 V5/V6，但 V6 的 Pixel F1-max 略高于 V4 (0.2834 对 0.2787)。

这说明在当前实验记录中，use_context_recentering 可能带来图像级判别与像素级定位之间的 trade-off。由于各版本除该开关外的参数变化尚未完整留档，上述结论仅是**观察性比较**，不能视为严格的单因素消融结论。后续应固定 V4 的全部训练设置，仅切换该开关，并在相同随机种子或多随机种子下重复实验，以完成可归因的消融验证。

6.7 强弱类别与误差分析

在最终 V4 的逐类别结果中，battery_tab2、ice_cream_stick、character_string 和 chip1 的像素级表现相对突出；其 Pixel AP / Pixel F1-max 分别为 0.6191 / 0.5995、0.5607 / 0.5573、0.4631 / 0.5121 和 0.4486 / 0.5248。相对而言，infusion_bottle_bottom5、nameplate7、resistance13 的 Pixel AP / Pixel F1-max 较低，分别为 0.0025 / 0.0087、0.0069 / 0.0339 和 0.0139 / 0.0450。

上述差异说明不同类别的定位难度不一致。仅凭汇总指标无法证明具体原因；异常区域面积、缺陷与背景的对比度、正常纹理的复杂程度等都可能相关，仍需结合异常图和对应真值掩膜进行定性核查。报告不将这些可能因素表述为已被实验直接证明的因果结论。

6.8 实验结论

六个版本的对比表明，图像级指标最高的版本不一定具有最佳的像素级定位能力。V4 采用项目默认超参数、未启用 use_context_recentering，在全局 Pixel AP (0.1190) 和 Pixel F1-max (0.2139) 上均为六个版本最佳，并在宏平均 Pixel AP (0.1987) 和 Pixel AUROC (0.8696) 上取得最高值。因此，在以像素级定位为主要目标的前提下，选择 V4 作为最终模型具有明确的数据依据。

7. 局限性与改进方向

1. 当前图像级分数以平滑后最高 1% 像素的均值计算。该汇聚策略对局部异常较敏感，但其比例是固定超参数；面对面积特别大或特别小的缺陷时，可进一步比较不同汇聚比例或自适应汇聚策略。

2. 输入采用固定 `Resize` 与中心裁剪。对于靠近原图边缘的细小异常，中心裁剪可能减少有效信息；后续可在保持输出尺寸要求的前提下评估保比例缩放、填充或多尺度推理。
3. 训练仅更新瓶颈与解码器，计算成本较低且具有较好的稳定性，但固定主干无法针对特定工业纹理进行适配。可在不破坏离线和复现要求的前提下探索参数高效微调或更合适的层融合策略。
4. `min-max` 图像级分数归一化依赖训练集分数范围。若训练与评测数据分布差异较大，归一化范围可能影响图像级分数的相对尺度；可比较鲁棒分位数归一化等替代方法。

8. 结论

本文给出了一个基于冻结 DINOv2 特征重建的工业异常检测方案。方案利用正常样本训练轻量瓶颈与解码器，通过多层编码—解码特征的余弦距离同时完成像素级定位和图像级判别。项目以 `manifest` 驱动数据访问，通过本地权重和模型文件支持离线加载，并按照 `sample_id` 输出连续的图像分数与 16-bit 像素异常图。

实验部分已为 5 组对比保留完整的指标填写位。补充实测数据后，建议结合定量结果、失败案例和运行资源记录，重点说明不同设置对 Pixel-level F1、Pixel-level AP 以及图像级判别能力的影响。